TILLÄMPAD ARTIFICIELL INTELLIGENS

F6 - DECISION MAKING: INTRO

FEEDBACK PÅ KOD

- Variabelnamn!
  - Okej om de är långa!
  - Här är mitt längsta variabelnamn i AI/TMU/Nätverk

```
std::unordered_map<GuaranteedNetworkMessageID, TimeType> myReceivedGuaranteedMessagesToLastReceivedTimestamp;
```

  - Desto större kodbaser blir, desto viktigare blir det att kommunicera funktionalitet och intention.
    - Blir ***mycket*** enklare att läsa system och att underhålla dem.
    - 9 av 10 gånger är det du själv som kommer tillbaka till ett system några veckor senare och ska lösa en bugg i det.
      - ”Vafaaan var nu int k = aThing igen…”
  - Förkortningar är farliga
    - Många som har döpt ”computer” till ”comp” i U03
    - ”comp som i component?” är det första jag tänker
    - Den sämsta förkortningen jag sett är chld, istället för child.
    - https://www.youtube.com/watch?v=arDJ0PSyumA

FEEDBACK PÅ KOD

- Lite mer angående läsbarhet, fast på en högre nivå.
  - Många som har stora GameWorld::Update-funktioner
  - Här är mina exempel från U01-U04:

```cpp
void GameWorld_Assignment1::Update()
{
    UpdateTargets();

#if !IS_RETAIL_BUILD
    UpdateImGui();
#endif // !IS_RETAIL_BUILD

    mySeeker.Update();
    myArriver.Update();
    myWanderer.Update();
    myWanderer.WrapAroundWorld();
}
```

```cpp
void GameWorld_Assignment2::Update()
{
    for (u32 i = 0; i < NR_OF_BOIDS; i++)
    {
        const HD_Vector2_f32 positionPreUpdate = myBoids[i].GetPosition();
        myBoids[i].Update();
        myBoids[i].WrapAroundWorld();
        const HD_Vector2_f32 positionPostUpdate = myBoids[i].GetPosition();

        myGrid.Update(&myBoids[i], positionPreUpdate, positionPostUpdate);
    }

    UpdateImGui();
}
```

```cpp
void GameWorld_Assignment3::Update()
{
    myPlayer.Update();
    mySmartPollGuard.Update();
    mySmartEventGuard.Update();
    myStupidPollGuard.Update();
    myStupidEventGuard.Update();

    UpdateLastClickedMousePosition();
    UpdatePlayerHacking();
}
```

```cpp
void GameWorld_Assignment4::Update()
{
    const HD_InputManager& inputManager = HD_InputManager::GetInstance();
    if (inputManager.GetIsKeyPressed(VK_LBUTTON))
    {
        HandleLMBPressed();
    }

    myPlayer.Update();
    myCompanion.Update();
}
```

FEEDBACK PÅ KOD

- Många av er som gör members till pekare när de inte behöver vara det.
  - T.ex:

```
class GameWorld
{
public:
    // public stuff

private:
    std::vector<Actor*> myActors;
    SeekController* mySeekController;
    ArriveController* myArriveController;
    WanderController* myWanderController;
};
```

  - Istället för:

```
class GameWorld
{
public:
    // public stuff

private:
    std::vector<Actor> myActors;
    SeekController mySeekController;
    ArriveController myArriveController;
    WanderController myWanderController;
};
```

EN SISTA GREJ

- Jag märker att jag ger många kompletteringar
  - Det är ok! Men förstår att det kan kännas tufft.

  - Räkna typ alltid med minst 1 komplettering.
    - Se det lite som code reviews i branschen.
    - Antalet gånger jag kunnat submitta något utan att rätta till det är noll.

    - Se inte komplettering som en fail. Det är något helt normalt, och ett del av vårt arbetsflöde som programmerare.
    - Hoppas det känns ok 🙂

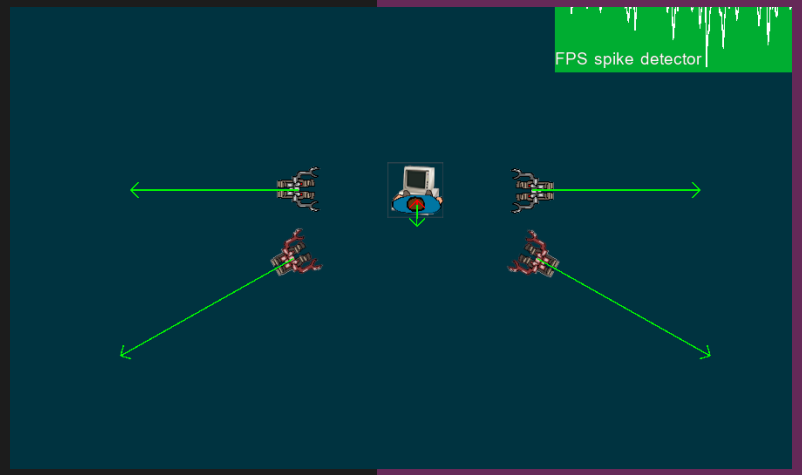
INTRO

- Än så länge har vi mest pratat om movement och navigering.
- Men vi har ännu inte pratat om det som gör att en AI ”känns smart”, i alla fall enligt spelare.
  - Det är beslutstagande!

- Ett väldigt grundläggande område inom AI.
  - Gör att vi kan börja bygga mer komplexa beteenden än de vi hittills har byggt med steering behaviours.

INTRO

- Så vad menar vi med beslutstagande?
  - Något händer -> AI:n reagerar
  - … men det gjorde vi ju i polling/event-uppgiften?
    - Spelaren hackar en dator, vakterna reagerar.



- Vi hade en controller som använde AIPollingStation eller AIEventManager.
  - Vi kan klara oss med det.

  - Men tänk er att vi har en rätt komplex AI, en boss kanske.
  - Blir en mega-chonky controller då.
  - Vi behöver system för att modulärt bygga upp ett beteende som scale:ar bättre.

DECISION MAKING

- Lite mer bakgrundsinfo innan vi snackar datastrukturer och kod.

- För att en AI ska kunna göra val måste det finnas saker att välja mellan.
  - Saker som den inte kan göra samtidigt.

  - En controller exekverar alltid samma kod varje frame, även om ***beteendet*** den resulterar i kan vara olika mellan olika frames.

  - De datastrukturerna vi ska kolla på kan exekvera olika klasser/cpp:s mellan olika frames. Viktig skillnad.
    - Det är det som gör dessa datastrukturer mer scalable.

DECISION MAKING

- Har nämnt innan, men här är lite repetition:
  - Tänk på att en AI inte gör val för sig själv, den gör val för spelaren.
  - Om ett val inte är synligt för spelaren så behövs det inte!

- De val eller states en AI har bör vara framåtblickande och goal oriented.
  - Mindre bra
    - Gå fram
    - Hoppa
  - Bättre
    - Leta upp ett health pack
    - Wander

DECISION MAKING

- Dessa kommandon kan brytas ner till mindre bitar som vi lämnar över till vår movement-kod.

- T.ex. ”Leta upp ett health pack”:
  - Gör en koll inom en radie och se om vi hittar ett health pack.
  - Kan vi gå dit?
  - Kör movement-algoritmer för att komma till den punkten.

- Men om det inte finns ett health pack nära AI:n?
  - Vi skulle kunna ge AI:n information om alla health packs i hela världen.
  - … och det för oss in på nästa punkt.

DECISION MAKING

- En stor fråga när vi jobbar med AI är:
  - Borde AI fuska? 🤔

- Om det gör spelupplevelsen bättre: Ja!
  - Spelare kan acceptera "fusk" så länge det skapar en kul upplevelse och det inte är övertydligt att AI:n fuskar.
  - Spel gör det här hela tiden!

DECISION MAKING

- T.ex Dark Souls:
  - Bossar läser av när spelaren healar.
  - (fusk eller bara rimligt...?)



DECISION MAKING

- Age of Empires 2
  - Fiende-AI får ofta resurser utan att villagers samlar dem.
  - AI på de högre svårighetsgraderna kan bygga byggnader snabbare än spelaren



DECISION MAKING

- Mario Kart
  - Spelarens nuvarande placering påverkar vilka items man får.
  - AI kör bättre/sämre beroende på om den ligger efter/före spelaren.
    - Kallas rubber banding
- Mario Party
  - Dice rolls för att flytta på sig är inte random.

- AI fuskar alltså inte alltid till sin egen fördel!





DECISION MAKING

- Enklaste sättet för en AI att fuska är att ge den information om hela världen.
  - Även saker som den inte borde kunna känna till.
    - Spelarens position (även om spelaren är bakom en vägg)
    - Alla health packs i världen
  - Spelaren märker inte om en AI har tillgång till all data, den märker bara hur AI:n agerar.
    - Det gör alltså inget om AI har tillgång till all data, så länge vi ser till att den agerar på den på ett smart sätt.
- Tillbaka till vårt ”Leta upp ett health pack”-exempel.
  - Om det inte finns ett health pack i AI:ns vision radius
    - Idle:a eller wander tills den hittar ett?
    - Gå till närmsta health pack, även om den inte ser det?
    - Kanske bara insta-heal:a om den är utanför spelarens vision och det var tillräckligt längesedan spelaren såg AI:n?
    - Vad vi vill göra beror kanske på AI:ns svårighetsgrad?

DECISION MAKING

- När vi väl har tänkt ut dataflödet mot AI:n och vilken info den känner till så kan vi börja skapa koden som är själva AI-algoritmen.

- Var bör vi börja?
  - Vilka val är viktigast för spelaren?
  - Vad kommer att ge spelet mest spelkänsla?
  - Hur får jag min AI att inte förlora?
  - Vilka actions ska vår AI kunna ta?

- Det viktigaste när vi implementerar en AI är att vi jobbar i kod-system som gör det enkelt både för design och kod att iterera.
  - Typ som med allt annat 🙃

DECISION MAKING

- Börja med AI:ns ”default state”.
  - Kanske är det wandering.
  - Kanske är det shooting och healing.

- Skapa enkla val. Börja med MVP.

  - Bryt upp er AI i de delar som ni vill att den ska kunna hantera. Kanske göra en lista med alla actions?

  - När vi har en ”default state” AI, implementera det som ger mest gameplay och interaktion.
    - AI som agerar på spelarens input eller position.
    - AI som försöker att inte förlora?
    - … vilket leder oss in på…

DECISION MAKING

- Två vanliga saker vi kan tänka på när vi implementerar en AI:
  - Se till att AI:n försöker att inte förlora
    - Spelaren interagerar med spelet, spelet reagerar tillbaka
  - Se till att AI:n försöker vinna
    - Om spelaren inte gör så mycket så skapar AI:n ett hot eller sätter press på spelaren.
  - Båda dessa tankesätt gör det tydligt för spelaren att det finns en konflikt, eller ett ”liv” i spelet.

DECISION MAKING

- Nu har vi i teorin en AI som kan spela mot spelaren!

- Vi vet vad AI:n behöver göra för att kunna vara en utmaning för spelaren.
  - Patrol, drink potion, cast lightning bolt, chase player
  - Etc...

- Hur strukturerar vi upp koden för vår decision making?
  - Speciellt så att den blir enkel att iterera.
  - Och enkel att scale:a upp utan att bli oöverskådligt komplex.

DECISION MAKING

- Det finns två huvudsakliga strukturer som brukar användas vid enkelare decision making:
  - Decision Tree
  - State Machine

- Ni har kanske t.o.m. redan gjort andra typer av state machines.
  - T.ex. en state machine för själva spelet.
    - Main menu, intro cinematic, in-game, credits, osv…

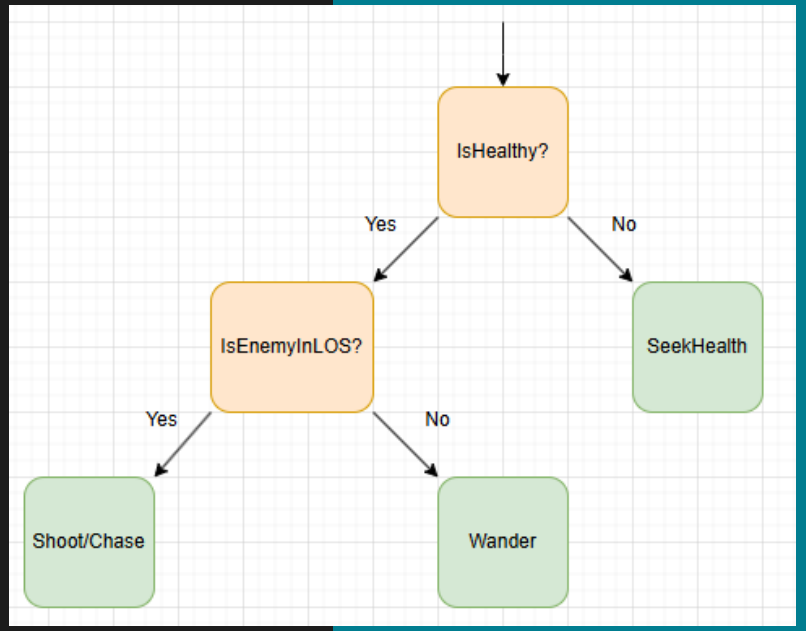
DECISION TREE

- Decision trees är glorifierade if-kedjor.
  - Jippie. 😐
  - Men de är fett användbara!

- Vi placerar våra val i en trädstruktur.

- Vi kollar på varje nod om vi ska åt höger eller vänster i trädet tills vi når en lövnod.

- Lövnoden är vårt beteende som vi ska köra.

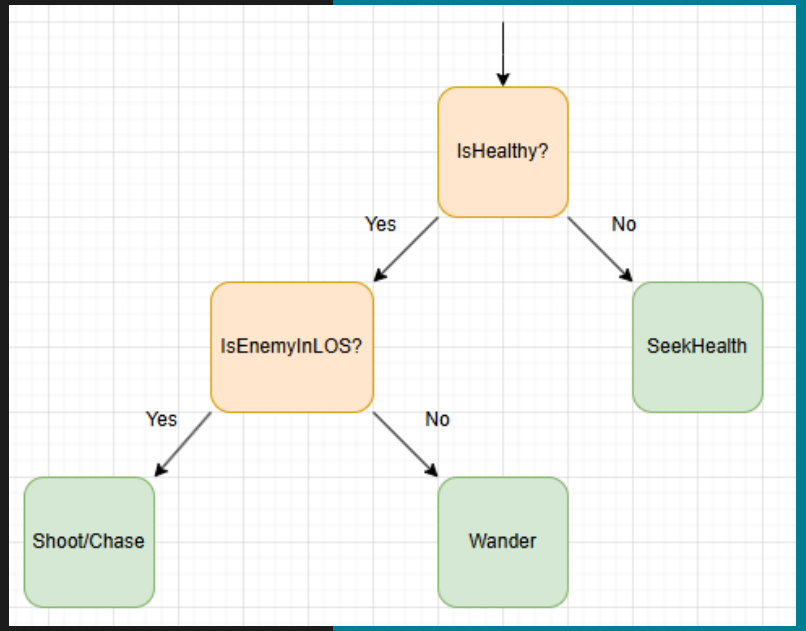


DECISION TREE

- Det finns två typer av noder:
  - Decision nodes
  - Action nodes

- Decision nodes returnerar sant eller falskt.
  - IsHealthy?
  - IsTargetInLOS?

- Action nodes innehåller ett beteende.
  - Gå till position (x, y)
  - Skjut
  - Drick potion



DECISION TREE

- Finns olika sätt att implementera ett decision tree. Här är ett förslag:
  - DecisionTreeNode

```
class DT_Node
{
public:
    DT_Node() {};
    virtual ~DT_Node() {};

    virtual void DeInit() {};

    virtual const DT_ActionNode* MakeDecision(const Actor& aActor) const = 0;

#if !IS_RETAIL_BUILD
    virtual void DebugDraw(const Actor& aActor) const { HD_Unused(aActor); };
#endif // !IS_RETAIL_BUILD
};
```

  - DecisionNode

```
class DT_DecisionNode : public DT_Node
{
public:
    DT_DecisionNode();

    void DeInit() override;

    void SetTrueNode(DT_Node* aTrueNode);
    void SetFalseNode(DT_Node* aFalseNode);

#if !IS_RETAIL_BUILD
    void DebugDraw(const Actor& aActor) const override;
#endif // !IS_RETAIL_BUILD

protected:
    DT_Node* myTrueNode;
    DT_Node* myFalseNode;
};
```

  - ActionNode

```
class DT_ActionNode : public DT_Node
{
public:
    DT_ActionNode();

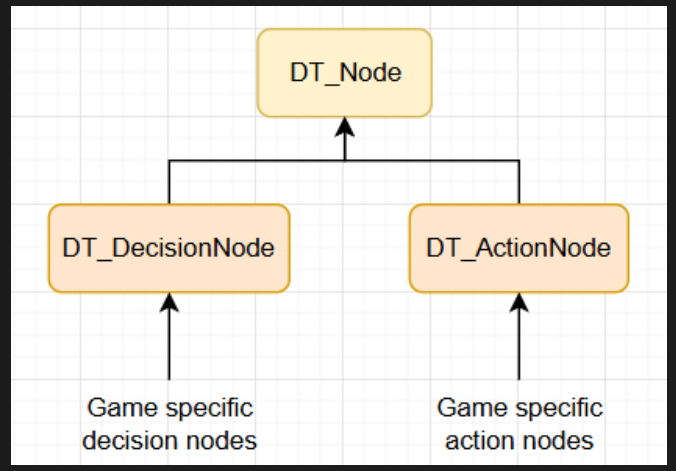
    const DT_ActionNode* MakeDecision(const Actor& aActor) const override;

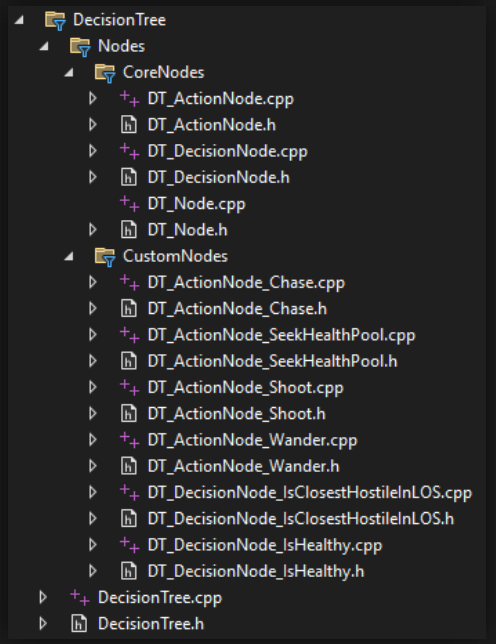
    virtual void UpdateActorControllerData(ActorControllerData& outActorControllerData) const;
    virtual HD_Vector2_f32 GetSteeringForce(const Actor& aActor) const;
    virtual void TriggerAction(Actor& aActor) const;

#if !IS_RETAIL_BUILD
    virtual Tga::Color DebugGetActionNodeColor() const { return { 1.f, 1.f, 1.f, 1.f }; }
#endif // !IS_RETAIL_BUILD
};
```

DECISION TREE

- Vi har alltså tre lager av arv





- Tricket ligger i DT_Node::MakeDecision
  - Children till DT_DecisionNode gör en check och call:ar MakeDecision på sin true eller false node.
  - Men alla ActionNodes delar samma MakeDecision-implementation, och den returnerar bara this. Vi ska snart se varför.

DECISION TREE

- DecisionTree

```
class DecisionTree
{
public:
    DecisionTree();

    void DeInit();

    const DT_ActionNode* MakeDecision(const Actor& aActor) const;

    void SetRoot(DT_Node* aRoot);

#if !IS_RETAIL_BUILD
    void DebugDraw(const Actor& aActor) const;
#endif // !IS_RETAIL_BUILD

private:
    DT_Node* myRoot;
};
```

- DecisionTreeActor

```
class DecisionTreeActor : public Actor
{
public:
    DecisionTreeActor();

    void SetDecisionTree(const DecisionTree* aDecisionTree);

private:
    void PreUpdate() override;
    void UpdateActorControllerData() override;
    void AddSteeringForce() override;
    void TriggerAction() override;

#if !IS_RETAIL_BUILD
    void UpdateDebugColor() override;
#endif // !IS_RETAIL_BUILD

    const DecisionTree* myDecisionTree;
    const DT_ActionNode* myActionNode;
};
```

- Actor

```
class Actor : public Entity
{
public:
    // Massa public grejs

protected:
    virtual void PreUpdate() {}
    virtual void UpdateActorControllerData() {}
    virtual void AddSteeringForce() {}
    virtual void TriggerAction() {}

    // Mer protected grejs

private:
    // Private grejs
};
```

```
void Actor::Update()
{
    PreUpdate();

    if (GetIsAlive())
    {
        // Update data needed to calculate new position
        UpdateActorControllerData();
        AddSteeringForce();

        // Update position
        UpdateMovement();

        // Calls dependent on position
        UpdateClosestHostile();
        UpdateHealing();
        TriggerAction();

#if !IS_RETAIL_BUILD
        UpdateDebugColor();
#endif // !IS_RETAIL_BUILD
    }
    else
    {
        UpdateRespawn();
    }
}
```

- I PreUpdate gör DecisionTreeActor myDecisionTree->MakeDecision och får en action node som den kan calla GetSteeringForce och TriggerAction på.
- MakeDecision i DecisionTree returnerar myRoot->MakeDecision

TGA

DECISION TREE

- GameWorld kan ha myDecisionTree
  - Sen kan man ha InitDecisionTree som allokerar noderna, sätter true och false nodes och sätter roten på myDecisionTree.

- Jag förstår om allt detta är överväldigande!
  - Hur hänger allt ihop egentligen?!
  - Vi har alla olika idéer om vilken arkitektur som känns mest rimlig.

DECISION TREE

- Använd min implementation om den ger er inspiration.
  - Annars:
    - Fråga mig, vi kan diskutera andra approaches.
    - Kolla boken
    - Diskutera med varandra
    - Börja smått, bygg upp nåt som ni tycker om att jobba med och som passar kodbasen ni har byggt upp hittills i kursen.
    - Gameplay-arkitektur är en skill som kommer med mycket mycket övning.
    - Var ska myDecisionTree finnas? Vem ”äger” det?
- Jag tog inspiration från kurslitteraturens implementering och ändrade lite.
- Ni måste inte koda er struktur exakt som mig eller som boken, så länge ni inte går bananas.

DECISION TREES

- Största skillnaden och fördelen mellan decision trees jämfört med if-satser är att trädet kan skapas dynamiskt.
  - Enklare att pussla ihop mer komplexa beteenden.
  - Man skulle kunna ha ett decision tree tool för design.
    - Då hade vi också sluppit pilla ihop trädet i kod.
  - Det kan också vara nice att decisions och actions isoleras till klasser.
    - Kan göra det enklare att få en översikt än att de ligger i funktioner.
    - Fast det hade säkert gått fint att implementera ett funktions-orienterat decision tree. Noderna håller ju inte egentligen i ett state/data. Vi är alla lite objektorienterings-skadade.🤔

TGA

DECISION TREE

- Decision trees körs vanligtvis varje frame från roten.
  - Varje frame börjar vi i rotnoden och rör oss neråt tills vi hittar en lövnod att exekvera.
  - Detta är en bland de största nackdelarna med decision trees.
    - Måste konstant göra checkar mot allt
    - Även fast vi gjorde samma checkar förra framen

- Det går att göra så att trädstrukturen kommer ihåg var den var senast.
  - Men inget krav att implementera det.
    - Vi kommer ändå kolla på det när vi går igenom Behaviour Tree

  - Man ska antingen ha sjukt många eller sjukt stora decision trees för att det ska vara en bottleneck att köra igenom dem från roten varje frame.

PAUS

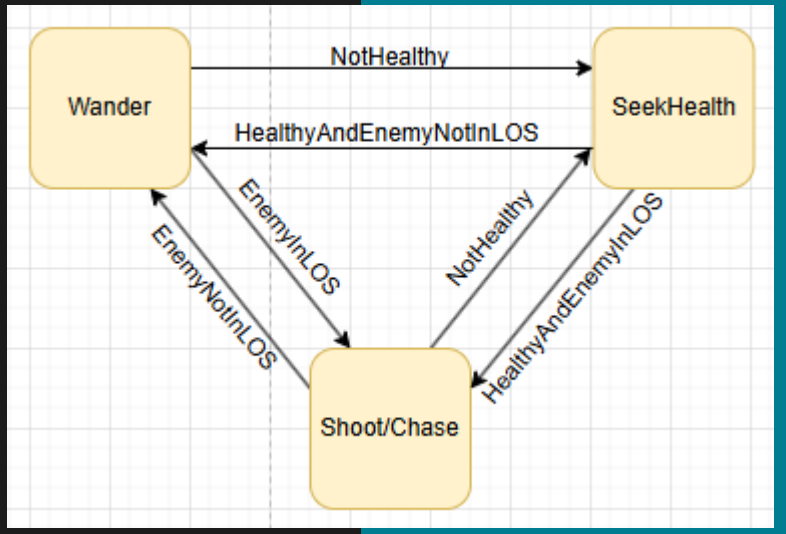


STATE MACHINE

- Förutom decision trees finns även state machines!

- Vi befinner oss i ett state och exekverar dess beteende tills något triggar att vi byter state.

- Varje state har transition conditions till ett eller flera andra states.





STATE MACHINE

- Såhär kan det se ut i kod!

```
class StateMachine
{
public:
    StateMachine();

    void DeInit();

    void AddState(State* aState);
    void SetInitialState(State* aInitialState);

    void CheckStateTransition(State*& inoutCurrentState, const Actor& aActor) const;

private:
    std::vector<State*> myStates;
    State* myInitialState;
};
```

STATE MACHINE

```cpp
typedef std::function<bool(const Actor&)> StateTransitionCondition;

struct StateTransitionData
{
    StateTransitionData(State* aStateToTransitionTo, StateTransitionCondition aStateTransitionCondition)
        : myStateToTransitionTo(aStateToTransitionTo)
        , myStateTransitionCondition(aStateTransitionCondition)
    {
    }

    State* myStateToTransitionTo = nullptr;
    StateTransitionCondition myStateTransitionCondition;
};

class State
{
public:
    State();
    virtual ~State() = default;

    virtual void OnEnterState() {}
    virtual void OnExitState() {}

    virtual void UpdateActorControllerData(ActorControllerData& outActorControllerData) const;
    virtual HD_Vector2_f32 GetSteeringForce(const Actor& aActor) const;
    virtual void TriggerAction(Actor& aActor) const;

    bool GetShouldTransition(State*& outStateToTransitionTo, const Actor& aActor);
    void AddTransitionState(State* aStateToTransitionTo, StateTransitionCondition aStateTransitionCondition);

#if !IS_RETAIL_BUILD
    virtual const char* DebugGetStateString() const = 0;
    virtual Tga::Color DebugGetStateColor() const { return { 1.f, 1.f, 1.f, 1.f }; }
#endif // !IS_RETAIL_BUILD

private:
    std::vector<StateTransitionData> myStateTransitionData;
};
```

STATE MACHINE

SeekHealthPool

```
class State_SeekHealthPool : public State
{
public:
    State_SeekHealthPool();

    HD_Vector2_f32 GetSteeringForce(const Actor& aActor) const override;

#if !IS_RETAIL_BUILD
    const char* DebugGetStateString() const override { return "SeekHealthPool"; }
    Tga::Color DebugGetStateColor() const override { return { 1.f, 0.f, 0.f, 1.f }; }
#endif // !IS_RETAIL_BUILD
};
```

Shoot

```
class State_Shoot : public State
{
public:
    State_Shoot();

    HD_Vector2_f32 GetSteeringForce(const Actor& aActor) const override;
    void TriggerAction(Actor& aActor) const override;

#if !IS_RETAIL_BUILD
    const char* DebugGetStateString() const override { return "Shoot"; }
    Tga::Color DebugGetStateColor() const override { return { 0.5f, 0.5f, 0.5f, 1.f }; }
#endif // !IS_RETAIL_BUILD

};
```

Wander

```
class State_Wander : public State
{
public:
    State_Wander(const WanderController* aWanderController, const WallAvoidanceController* aWallAvoidanceController);

    void UpdateActorControllerData(ActorControllerData& outActorControllerData) const override;
    HD_Vector2_f32 GetSteeringForce(const Actor& aActor) const override;

#if !IS_RETAIL_BUILD
    const char* DebugGetStateString() const override { return "Wander"; }
    Tga::Color DebugGetStateColor() const override { return { 0.f, 1.f, 0.f, 1.f }; }
#endif // !IS_RETAIL_BUILD

private:
    const WanderController* myWanderController;
    const WallAvoidanceController* myWallAvoidanceController;
};
```

STATE MACHINE

- Samma sidenote som decision tree...
  - Finns många sätt att implementera en state machine!
  - Kolla på min kod, kolla i kurslitteraturen, snacka med varandra, implementera vad som känns rimligt för er.

  - Jag gjorde en poll-baserad state machine, men det går också bra att göra den event-baserad.
    - Kanske events t.o.m. är bättre för detta?

    - Polling vs events i state machine, say what?
      - Mer om detta om ett par slides.

STATE MACHINES

- Fördel med state machine?
  - Vi exekverar inte massa if-satser varje frame som i ett decision tree.
    - (Nja, nånstans måste vi ju exekvera if-satser om vi ska transition:a till ett nytt state. Hur många beror på hur många transition states vi har.)
  - Vi är i vårt nuvarande state och kör det tills vi transition:ar till vårt nästa state.

- Nackdel?
  - State machines kan bli jobbiga att underhålla när det finns många states.
  - Det är ok när det är få states, men..
    - Om vi redan har många states och vill lägga till ett nytt state så måste vi kolla vilka transitions som ska finnas till och från alla andra states.
    - I ett decision tree behöver vi bara koda en ny action node och lägga in den i trädet där den hör hemma.

- Vi vill helst inte ha jättemånga transitions i en state machine, utan behålla state machinen så streamlined som möjligt.

TGA

STATE MACHINES

- Det är möjligt att kombinera state machines med decision trees!
  - Beror ofta på vad som är smidigt att använda för AI-designers.
  - Man kan tänka sig att en AI har en state machine som bestämmer dess ”macro”-beteende.
    - Chase, Flee, Wander, etc...
  - Och sen kan varje state innehålla ett decision tree som går in i mer detalj på beteendet.
  - Exakt hur man strukturerar upp AI är alltid ett samarbete mellan olika dicipliner.
    - Ofta design och kod

SAMMANFATTNINGSVIS

- Decision trees har noder och state machines har states, ait...
  - ... men, liksom, hur hänger allt ihop då?
  - Var och när call:ar vi GetSteeringForce och TriggerAction?



SAMMANFATTNINGSVIS, ARKITEKTUR-RECAP

```cpp
class Actor : public Entity
{
public:
    // Massa public grejs

protected:
    virtual void PreUpdate() {}
    virtual void UpdateActorControllerData() {}
    virtual void AddSteeringForce() {}
    virtual void TriggerAction() {}

    // Mer protected grejs

private:
    // Private grejs
};
```

```cpp
void Actor::UpdateMovement()
{
    const f32 deltaTime = Tga::Engine::GetInstance()->GetDeltaTime();

    mySteeringForce.Limit(myMaxForce);
    myAcceleration = mySteeringForce /* / myMass */;

    myVelocity += (myAcceleration * 0.5f) * deltaTime;
    myVelocity.Limit(myMaxSpeed);

    constexpr f32 standStillSpeed = 0.01f;
    if (myVelocity.Length2() > (standStillSpeed * standStillSpeed))
    {
        myPosition += myVelocity * deltaTime;
    }

    myVelocity += (myAcceleration * 0.5f) * deltaTime;
    myVelocity.Limit(myMaxSpeed);

    myRotation = atan2f(myVelocity.myY, myVelocity.myX);
    mySteeringForce.Set(0.f, 0.f);
}
```

```cpp
void Actor::Update()
{
    PreUpdate();

    if (GetIsAlive())
    {
        // Update data needed to calculate new position
        UpdateActorControllerData();
        AddSteeringForce();

        // Update position
        UpdateMovement();

        // Calls dependent on position
        UpdateClosestHostile();
        UpdateHealing();
        TriggerAction();

#if !IS_RETAIL_BUILD
        UpdateDebugColor();
#endif // !IS_RETAIL_BUILD
    }
    else
    {
        UpdateRespawn();
    }
}
```

- AddSteeringForce gör GetSteeringForce på en action node eller ett state och lägger till force:en på mySteeringForce.

SAMMANFATTNINGSVIS

```cpp
void DecisionTreeActor::PreUpdate()
{
    myActionNode = myDecisionTree->MakeDecision(*this);
}

void DecisionTreeActor::UpdateActorControllerData()
{
    myActionNode->UpdateActorControllerData(myActorControllerData);
}

void DecisionTreeActor::AddSteeringForce()
{
    const HD_Vector2_f32 steeringForce = myActionNode->GetSteeringForce(*this);
    AddForce(steeringForce);
}

void DecisionTreeActor::TriggerAction()
{
    myActionNode->TriggerAction(*this);
}
```

```cpp
class DecisionTreeActor : public Actor
{
public:
    DecisionTreeActor();

    void SetDecisionTree(const DecisionTree* aDecisionTree);

private:
    void PreUpdate() override;
    void UpdateActorControllerData() override;
    void AddSteeringForce() override;
    void TriggerAction() override;

#if !IS_RETAIL_BUILD
    void UpdateDebugColor() override;
#endif // !IS_RETAIL_BUILD

    const DecisionTree* myDecisionTree;
    const DT_ActionNode* myActionNode;
};
```

- Sen kan man tänka sig något liknande för StateMachineActor.

SAMMANFATTNINGSVIS

```
void GameWorld_Assignment5::InitDecisionTree()
{
    // Allocate decision and action nodes

    // Set true nodes and false nodes

    // Set root node on myDecisionTree
}

void GameWorld_Assignment5::InitStateMachine()
{
    // Allocate states

    // Set transition states

    // Add states to myStateMachine and set its initial state
}
```

- Exempel:
  - stateWander ska transition:a till stateShoot om closest hostile är i line of sight.

```
stateWander->SetTransitionState(stateShoot, [](const Actor& aActor)
{
    const bool isClosestHostileInLOS = aActor.GetIsClosestHostileInLOS();
    return isClosestHostileInLOS;
});
```

- Agående att vi sätter transition states här:
  - Det är det jag menade med att denna implementationen bygger på polling och inte på events.
  - Man kan istället t.ex. tänka sig att en state-klass har privata funktioner som hanterar events.
  - Upp till er hur ni vill göra!

SAMMANFATTNINGSVIS

- När ska vi välja decision tree eller state machine?

- Det beror på:
  - Hur ni vill att flödet mellan beteenden ska se ut
    - Är det viktigt för er att ha koll på specifika transitions?
      - Om ja: state machine. Annars, decision tree.

  - Men kanske viktigast av allt
    - Vad AI-design föredrar att jobba med

- En rough bottom line
  - Använd decision tree som goto för riktigt simpel AI.
    - Går snabbt att sätta upp
    - Låg komplexitet
    - Så länge trädet inte är stort så är det inte dyrt att köra dess if-satser.
  - Använd state machine om AI:n är aningen mer komplex.
  - ...men...

SAMMANFATTNINGSVIS

- Använd ingen av dem för komplicerad AI!
- Inte rekommenderat att ni skapar monoliter för AI:ns beteende.
  - Alltså ett enormt decision tree
  - Eller en gigantisk state machine
  - Nästa föreläsning kommer vi kolla på en datastruktur för mer komplicerad AI.

- Som sagt: vi behöver inte välja, vi kan istället kombinera.
  - En state machine som väljer decision tree
    - Kallas HSM (hierarchical state machine. Kommer inte gå igenom detta nu men intressant ämne ifall man vill kolla.)
  - Enter Boss Phase 2 State
    - Run Decision Tree for Boss Phase 2

SAMMANFATTNINGSVIS

- Flödet i AI kan väldigt snabbt bli väldigt komplext
  - Kan vara svårt att veta vad det är som händer när vi debuggar i en lövnod i ett träd som är i en state machine som är i en lövnod…
- I större motorer är det vanligt att vi använder oss av andra strukturer och begrepp
  - Behavior Trees!
  - Fuzzy Logic
  - Goal-Oriented Action Planning (GOAP)
- Mer om detta framöver!

KURSLITTERATUR

- Denna föreläsningen
  - Chapter 5 - Decision Making
    - 5.1 Overview of Decision Making (s 297-299)
    - 5.2 Decision Trees (s 299-314)
    - 5.3 State Machines (s 314-337)

- Nästa föreläsning (på onsdag!)
  - Chapter 5 - Decision Making
    - 5.5 Fuzzy Logic (s 374-399)
    - 5.6 Markov Systems (s 399-405)
    - 5.7 Goal-Oriented Behavior (s 405-431)
  - Chapter 10 - Execution Management
    - 10.3 Level of Detail (s 821-834)
  - Extra:
    - Chapter 5 - Decision Making
      - 5.8 Rule-Based Systems (s 431-461)
      - 5.9 Blackboard Architectures (s 461-468)
      - 5.10 Action Execution (s 469-483)
    - Chapter 6 - Tactical and Strategic AI
      - 6.1 Waypoint tactics (s 485-511)
      - 6.2 Tactical Analyses (s 511-547)
      - 6.3 Tactical Pathfinding (s 548-553)

UPPGIFTEN

- Vi ska göra robot wars!



- Två robotar (som inte måste vara robotar)
  - En som styrs av ett Decision Tree och en som styrs av en State Machine.
  - De ska ha samma beteende som är följande:

UPPGIFTEN

- Era actors ska undvika att köra in i och kollidera med väggar.
  - Världen behöver inte vara komplicerad, ok om den bara är en tom rektangel.
- Robotarna ska ha HP
  - Om deras HP faller under en viss gräns så anses de vara ”wounded” och ska då söka sig till en health pool som finns i leveln.
  - När de står nära health pool:en så ska de gradvis få tillbaka HP.
  - Det ska gå att se hur mycket HP en robot har, dvs HP ska renderas på något sätt.
- Om de inte är wounded så ska de kolla om de har vision på den andra roboten.
  - Om de inte har det så ska de bara vandra/idle:a omkring i leveln.
  - Om de har vision så ska de jaga och försöka skjuta den andra roboten.
  - Ni kan implementera vision väldigt enkelt, kan bara vara en distance check t.ex.
  - Metrics som bullet speed, fire rate, bullet spread etc är upp till er. Tänk på hur dessa kan påverka inte bara balansen i spelet utan även vilken känsla AIn ger spelaren!
  - Robotarna måste göra mer skada än vad health poolen kan heala. Annars kan ni lätt hamna i ett dödläge där båda actors står i health poolen och skjuter på varandra men ingen tar skada.
- När en actor dör så ska den respawna efter ett par sekunder.
- Om ni vill så får ni göra en mer komplicerad värld som har mer geometri och ha mer komplicerad vision check, typ ray casts.
- Ni får också ha mer komplicerat beteende på era AIs om ni vill. Så som uppgiften är beskriven är bara minimum. Men det är ändå ganska mycket att implementera. Därför ska ni få lite…

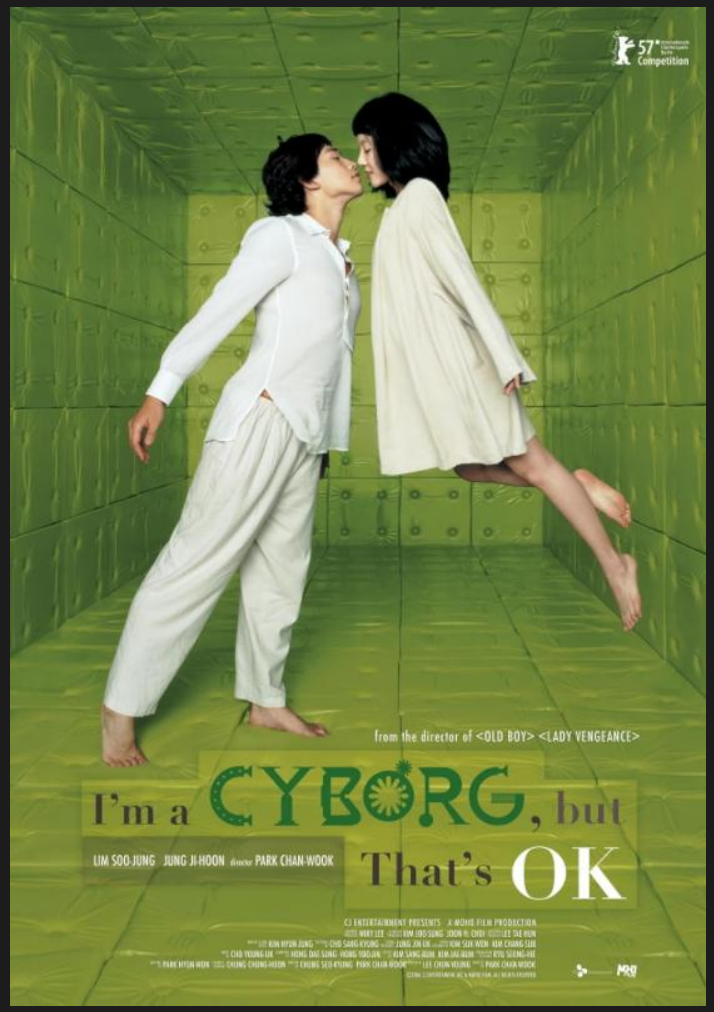
UPPGIFTEN

- Implementerings-tips!
  - Börja med roboten som har ett Decision Tree.
  - Implementera en feature i taget, t.ex implementera först dess normal-läge, dvs wandering + wall avoidance.
  - Implementera sedan HP och health poolen.
    - Innan ni har två robotar, gör så att en robot kan ta skada via någon hårdkodad hotkey, t.ex spacebar.
  - När det är klart kan ni implementera shooting.
    - Då kan ni fixa så att State Machine actorn finns i leveln men inte gör någonting, bara så att Decision Tree actorn har ett target.
  - När Decision Tree actorn är klar kan ni ”disable:a” den så att den finns i leveln men inte gör någonting och börja implementera State Machine actorn.
  - När båda era actors är klara, prova att enable:a båda och se dem battle it out. >:)

- (reminder to self: visa exempel)

DAGENS AI-FILMTIPS

- I’m a cyborg, but that’s OK (2006), Park Chan-Wook



FRÅGOR?

